MODUL PRAKTIKUM: OBJECT-ORIENTED PROGRAMMING
MINGGU 6: Interfaces & Code Contracts

Waktu Pengerjaan: 100 menit (Lab Session)

Kata-kata pembuka:
"Interface itu ibarat kontrak kerja yang sangat ketat. Ia mendefinisikan apa yang bisa kamu lakukan (Can-Do) tanpa peduli siapa kamu sebenarnya (Is-A). Kalau kamu masih bingung membedakan Interface dan Abstract Class, ingat ini: Abstract class menentukan identitas utamamu (kamu hanya bisa punya satu Bapak) , sedangkan Interface adalah skill atau kemampuan tambahanmu (kamu bisa menguasai banyak skill). Hari ini kita akan melakukan banyak trial and error, menembus batas single inheritance, dan menyelesaikan kebingungan compiler yang dikenal sebagai The Diamond Problem. Pastikan jarimu siap untuk commit!"

A. ATURAN PROJECT (REMINDER)

Ingat kebijakan Single Repository dari minggu lalu. Jangan buat project baru!

1. Buka project `IF433_Lab_Works` yang sudah dibuat minggu lalu.
2. Di dalam folder `src`, buat package baru `oop_<nim>_<nama>.week06`.
3. Pastikan semua kode hari ini masuk ke package `week06`.

B. LATIHAN TERBIMBING 1: Stateless Interface & No Backing Field (Trial & Error)

Tujuan: Memahami aturan *Implicit Abstraction* dan larangan penyimpanan *state* (Backing Field) di dalam Interface.

LANGKAH 1: Deklarasi Interface dengan Error Sengaja

Interface itu sepenuhnya *stateless*, artinya mereka tidak bisa menyimpan memori atau variabel. Mari kita buktikan.

1. Buat file `Clickable.kt`.
2. Ketik kode berikut. Perhatikan garis merah (*Compiler Error*) di bagian variabel `name`.

```kotlin
interface Clickable {
    // ERROR: Property in an interface cannot have a backing field
    val name: String = "Tombol Rahasia"

    // Function without body (Implicitly Abstract)
    fun click()
}
```

CHECKPOINT 1: *Commit* kode yang masih *error* ini dengan pesan:

```
"week06: (trial) create Clickable interface with backing field error"
```

LANGKAH 2: Memperbaiki Error (Abstract Property)

Alih-alih menyimpan *state*, Interface menuntut class turunannya yang menyediakan data tersebut. Hapus nilai bawaannya agar menjadi *Abstract Property*.

```
interface Clickable {
    val name: String // Abstract property, tidak ada nilai default
    fun click()
}
```

CHECKPOINT 2: *Commit* dengan pesan:

```
"week06: fix Clickable interface using abstract property"
```

LANGKAH 3: Implementasi Kontrak

Buat class `Button` yang menandatangani kontrak `Clickable`. Gunakan titik dua (`:`).

```
class Button(override val name: String) : Clickable {
    override fun click() {
        println("Tombol '$name' berhasil diklik!")
    }
}
```

CHECKPOINT 3: Commit dengan pesan:

```
"week06: create Button class implementing Clickable"
```

C. LATIHAN TERBIMBING 2: Interface vs Abstract Class (Compound Capabilities)

Tujuan: Mengatasi kelemahan *Single Inheritance* dimana sebuah class hanya bisa mewarisi satu *Parent Class*. Kita akan menggabungkan Abstract Class dan beberapa Interface secara bersamaan.

LANGKAH 1: Membuat Identitas Utama (Abstract Class)

1. Buat file `Watch.kt`.

```
// Abstract Class = Is-A (Core Identity)
abstract class Watch {
    abstract fun showTime()
}
```

CHECKPOINT 4 (COMMIT KE-4): *Commit* dengan pesan:

```
"week06: create Watch abstract class"
```

LANGKAH 2: Membuat Kemampuan Tambahan (Interfaces)

1. Buat file `Capabilities.kt` dan masukkan dua interface ini:

```
// Interface = Can-Do (Behavior)
interface BluetoothConnectable {
    fun connectToBluetooth()
}

interface Rechargeable {
    fun chargeBattery()
}
```

CHECKPOINT 5: *Commit* dengan pesan:

```
"week06: create compound interfaces"
```

LANGKAH 3: Multiple Implementation

Sebuah class bisa menandatangani banyak kontrak sekaligus. Ingat, *Parent Class* harus ditulis pertama, baru diikuti *Interfaces* menggunakan koma.

Buat file `Smartwatch.kt`.

```
// Menggabungkan 1 Parent Class dan 2 Interfaces
class Smartwatch : Watch(), BluetoothConnectable, Rechargeable {
    override fun showTime() {
        println("Layar OLED menyala: 14:00 WIB")
    }

    override fun connectToBluetooth() {
        println("Mencari perangkat HP di sekitar untuk pairing...")
    }

    override fun chargeBattery() {
        println("Mengisi daya menggunakan charger magnetik 15W.")
    }
}
```

CHECKPOINT 6: *Commit* dengan pesan:

```
"week06: create Smartwatch with multiple implementations"
```

D. LATIHAN TERBIMBING 3: Resolving Conflicts & Default Methods (The Diamond Problem)

Tujuan: Modern Kotlin mengizinkan Interface memiliki metode dengan *body* (Default Behavior). Apa yang terjadi jika dua interface punya fungsi dengan nama yang sama persis?

<u>LANGKAH 1: Membuat Interface yang Berkonflik</u>

Buat file `GadgetInterfaces.kt`:

```
interface Camera {
    fun turnOn() { println("Lensa kamera terbuka dan sensor aktif.") }
}

interface Phone {
    fun turnOn() { println("Sinyal seluler mencari jaringan.") }
}
```

CHECKPOINT 7: *Commit* dengan pesan:

```
"week06: create Camera and Phone interfaces with default methods"
```

LANGKAH 2: Memicu Diamond Problem (Trial)

Buat file `Smartphone.kt`. Ketik ini dan lihat garis merahnya. Compiler kebingungan implementasi mana yang harus dipakai.

```
// ERROR: Class Smartphone inherits multiple implementations of turnOn()
class Smartphone : Camera, Phone {
}
```

CHECKPOINT 8: *Commit* dengan pesan:

```
"week06: (trial) trigger the diamond problem compiler error"
```

LANGKAH 3: Resolusi Konflik Menggunakan `super`

Perbaiki `Smartphone.kt` dengan melakukan *explicit overriding*. Gunakan kurung siku (`< >`) untuk memilih implementasi mana yang mau dipanggil.

```
class Smartphone : Camera, Phone {
    // Manually override to resolve ambiguity
    override fun turnOn() {
        super<Camera>.turnOn() // Menjalankan logika Camera
        super<Phone>.turnOn()  // Menjalankan logika Phone
        println("Sistem operasi Smartphone berhasil booting.")
    }
}
```

CHECKPOINT 9: *Commit* dengan pesan:

```
"week06: fix diamond problem using explicit super delegation"
```

E. LATIHAN TERBIMBING 4: Decoupling & Interface as Data Type

Tujuan: Menggunakan interface sebagai tipe data parameter untuk memicu *Dynamic Dispatch* dan *Runtime Polymorphism*. Ini menciptakan arsitektur "*Plug-and-Play*" seperti blok Lego.

LANGKAH 1: Buat Kontrak E-Commerce

Buat file `PaymentMethod.kt` (sesuai *Case Study* di *slide*).

```
interface PaymentMethod {  5 Usages
    fun pay(amount: Double)  1 Usage
}
```

CHECKPOINT 10: *Commit* dengan pesan:

```
"week06: create PaymentMethod contract"
```

LANGKAH 2: Buat Implementasi

Buat class `Gopay` dan `CreditCard` di file `PaymentImplementations.kt`.

```
class Gopay : PaymentMethod {
    override fun pay(amount: Double) { println("Processing Rp$amount via Gopay Server") }
}

class CreditCard : PaymentMethod {
    override fun pay(amount: Double) { println("Contacting Bank for Rp$amount") }
}
```

CHECKPOINT 11: *Commit* dengan pesan:

```
"week06: create Gopay and CreditCard"
```

LANGKAH 3: Buat Core System & Testing di Main

Buat file `Main.kt`. Kita gunakan `PaymentMethod` sebagai tipe data parameter.

```
// Fungsi ini Decoupled! Tidak peduli kelas aslinya apa.
fun processCheckout(method: PaymentMethod, amount: Double) {
    println("-> Memulai checkout...")
    method.pay(amount) // Dynamic polymorphism in action
}
```

```
fun main() {
    val myWatch = Smartwatch()
    myWatch.showTime()

    val myPhone = Smartphone()
    myPhone.turnOn()

    val pay1 = Gopay()
    val pay2 = CreditCard()

    println("\n=== TESTING CHECKOUT ===")
    processCheckout( method = pay1,  amount = 50000.0)
    processCheckout( method = pay2,  amount = 150000.0)
}
```

CHECKPOINT 12: *Commit* dengan pesan:

```
"week06: implement decoupled processCheckout in main"
```

F. TUGAS MANDIRI: Sistem Smart Home Terintegrasi

Skenario: Kita akan membangun sistem *Smart Home* modular. Rumah modern tidak hanya punya lampu, tapi juga perangkat keamanan canggih (seperti ekosistem Tapo/Ezviz) dan *smart speaker* (seperti Google Nest). Karena perangkat-perangkat ini diproduksi pabrik yang berbeda, kita harus menyatukan mereka menggunakan *Interfaces*.

Langkah-langkah Tugas:

1. **Buat File `SmartInterfaces.kt`:**
   - Buat interface `SmartDevice`. Wajibkan ada abstract property `id: String` dan `name: String`.
   - Buat interface `Switchable`. Punya abstract function `turnOn()` dan `turnOff()`.

- Buat interface `Recordable`. Punya abstract function `startRecord()` dan default function `stopRecord()` yang mencetak "Perekaman dihentikan dan disimpan ke Cloud."

CHECKPOINT 13: *Commit* dengan pesan:

```
"week06: (task) create smart home interfaces"
```

2. **Buat File `SmartLamp.kt`**:
   - Mewarisi `SmartDevice` dan `Switchable`.
   - Implementasikan seluruh metodenya (cetak teks deskriptif saat `turnOn`/`turnOff` menggunakan namanya).

CHECKPOINT 14: Commit dengan pesan:

```
"week06: (task) create SmartLamp"
```

3. **Buat File `SmartSpeaker.kt`**:
   - Mewarisi `SmartDevice` dan `Switchable`.
   - Tambahkan fungsi spesifik `fun playMusic(song: String)` yang mencetak "Memutar lagu $song dari Spotify."

CHECKPOINT 15: Commit Main.kt dengan pesan:

```
"week06: (task) create SmartSpeaker"
```

4. **Buat File `SmartCCTV.kt`**:
   - Mewarisi `SmartDevice`, `Switchable`, DAN `Recordable` (Compound Capabilities!).
   - Saat `turnOn()`, selain menyala, wajib memanggil fungsi `startRecord()` secara otomatis dari dalam metodologinya.

CHECKPOINT 16: Commit Main.kt dengan pesan:

```
"week06: (task) create SmartCCTV with compound interfaces"
```

5. Buat File `SmartHomeHub.kt` (Sistem Core):
   - Buat class `SmartHomeHub`.
   - Miliki properti `val devices = mutableListOf<SmartDevice>()`.
   - Buat fungsi `addDevice(device: SmartDevice)`.
   - Buat fungsi `turnOffAllSwitches()`. Lakukan iterasi `for` ke `devices`. Gunakan **Smart Casting** (`is`) untuk mendeteksi: Jika *device* tersebut adalah sebuah `Switchable`, panggil `turnOff()`.

CHECKPOINT 17: Commit Main.kt dengan pesan:

```
"week06: (task) create SmartHomeHub logic"
```

   - Buat fungsi `activateSecurityMode()`. Lakukan iterasi. Jika *device* terdeteksi sebagai `Recordable`, panggil `startRecord()`. Jika terdeteksi sebagai `SmartSpeaker`, lakukan *casting* dan panggil `playMusic("Sirine Peringatan")`.

CHECKPOINT 18: Commit Main.kt dengan pesan:

```
"week06: (task) implement advanced smart casting in Hub"
```

6. Testing di `Main.kt`:
   - Instansiasi 1 `SmartLamp` ("Ruang Tamu"), 1 `SmartSpeaker` ("Google Nest Dapur"), dan 1 `SmartCCTV` ("Ezviz Garasi").

CHECKPOINT 19: Commit Main.kt dengan pesan:

```
"week06: (task) instantiate smart devices in main"
```

   - Instansiasi `SmartHomeHub`, tambahkan semua perangkat tadi.
   - Panggil `hub.activateSecurityMode()` dan `hub.turnOffAllSwitches()`. Pastikan *output*-nya dinamis sesuai perangkatnya!

CHECKPOINT 20: Commit Main.kt dengan pesan:

```
"week06: (task) test smart home system logic in main"
```

G. PENGUMPULAN

1. **Push ke GitHub:** Pastikan 20 Checkpoint (Commit) di atas sudah terkirim.
   git push
2. **Validasi**: Cek tab "Commits" di repository GitHub Anda. Riwayat pengerjaan *trial and error*, pemecahan Diamond Problem, hingga sistem *Smart Home* harus terlihat.
3. **Submit:** Copy Link Repository ke e-learning UMN (sudah dilakukan di minggu pertama)

Kata-kata penutup:

"Bagaimana rasanya menandatangani banyak kontrak sekaligus? Kalau programmu hari ini banyak error di Diamond Problem, itu wajar. Setidaknya sekarang kamu tahu bahwa punya dua sifat dari sumber berbeda butuh resolusi eksplisit, bukan sekadar kompromi diam-diam. Dan untuk kalian yang berhasil menyambungkan CCTV dan Speaker dalam satu Hub lewat polymorphism, selamat! Kalian sudah menguasai fondasi arsitektur modern yang decoupled. Jangan lupa push kodenya!"